Lab 16 – Database Design and Queries

Relational Schema Design, Normalization, CRUD & Query Optimization

Name: D.POORNANDA CHALAM **Roll No:** 2503A52361 **Batch:** 13

Task 1 – Design Database Schema for Student Management System

Question / Objective:
Design Database Schema for Student Management System with 3 tables (Students, Courses, Enrollments) following 1NF, 2NF, and 3NF.

AI Prompt:
"Generate SQL CREATE TABLE statements for a Student Management System with three tables: Students, Courses, and Enrollments. Include appropriate data types, primary keys, foreign keys, and constraints. Ensure the database design follows 1NF, 2NF, and 3NF. Keep the SQL code simple and compatible with MySQL."

Source Code:

```
-- Create Database
CREATE DATABASE StudentManagement;
USE StudentManagement;

-- Create Students Table
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);

-- Create Courses Table
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT
);

-- Create Enrollments Table
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Terminal Output (VS Code Execution):

```
mysql> CREATE DATABASE StudentManagement;
Query OK, 1 row affected (0.032 sec)
```

```
mysql> USE StudentManagement;
Database changed (0.000 sec)

mysql> CREATE TABLE Students (...);
Query OK, 0 rows affected (0.062 sec)

mysql> CREATE TABLE Courses (...);
Query OK, 0 rows affected (0.016 sec)

mysql> CREATE TABLE Enrollments (...);
Query OK, 0 rows affected (0.031 sec)

mysql> SHOW TABLES;
+------------------------------+
| Tables_in_studentmanagement  |
+------------------------------+
| courses                      |
| enrollments                  |
| students                     |
+------------------------------+
3 rows in set (0.015 sec)
```

Justification & Technical Explanation:
The schema is structured into three normalized relations: Students, Courses, and Enrollments. Each table has an atomic primary key satisfying 1NF. Non-key attributes fully depend on their entire primary key (2NF). Transitive dependencies are eliminated (3NF) by isolating student contact details and course descriptions into their respective entities, linked via foreign keys in the Enrollments junction table.

Task 2 – Insert Sample Records

Question / Objective:
Insert sample records into Students (>=3), Courses (>=4), and Enrollments (>=5), and verify with SELECT.

AI Prompt:
"Generate SQL INSERT statements to insert at least 3 students, 4 courses, and 5 enrollment records into the Students, Courses, and Enrollments tables. Also generate SELECT queries to verify the inserted records."

Source Code:

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES
(1, 1, 101, '2026-11-01'),
(2, 1, 102, '2026-11-01'),
(3, 2, 103, '2026-11-02'),
```

```
(4, 3, 101, '2026-11-02'),
(5, 3, 104, '2026-11-03');
```

Terminal Output (VS Code Execution):

```
mysql> SELECT * FROM Students;
+------------+-------+-------------------+
| student_id | name  | email             |
+------------+-------+-------------------+
|          1 | Arjun | arjun@gmail.com   |
|          2 | Meera | meera@gmail.com   |
|          3 | Rahul | rahul@gmail.com   |
+------------+-------+-------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Courses;
+-----------+--------------------+---------+
| course_id | course_name        | credits |
+-----------+--------------------+---------+
|       101 | Database Systems   |       4 |
|       102 | Data Structures    |       3 |
|       103 | Operating Systems  |       4 |
|       104 | AI Fundamentals    |       3 |
+-----------+--------------------+---------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollments;
+---------------+------------+-----------+-----------------+
| enrollment_id | student_id | course_id | enrollment_date |
+---------------+------------+-----------+-----------------+
|             1 |          1 |       101 | 2026-11-01      |
|             2 |          1 |       102 | 2026-11-01      |
|             3 |          2 |       103 | 2026-11-02      |
|             4 |          3 |       101 | 2026-11-02      |
|             5 |          3 |       104 | 2026-11-03      |
+---------------+------------+-----------+-----------------+
5 rows in set (0.00 sec)
```

Justification & Technical Explanation:

Sample records were populated enforcing referential integrity. Foreign key references in Enrollments correctly map to existing primary keys in Students and Courses. Verification SELECT statements confirm that all records were properly committed.

Task 3 – Perform CRUD Operations

Question / Objective:
Perform CRUD operations: retrieve all courses, update a student's email, delete an enrollment record, and verify using SELECT.

AI Prompt:
"Generate SQL queries to perform CRUD operations on the Student Management Database. Retrieve all courses, update a student's email, delete an enrollment record, and verify the changes using SELECT statements."

Source Code:

```
-- Read: Retrieve all courses
SELECT * FROM Courses;

-- Update: Modify student's email
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;

-- Delete: Remove an enrollment record
DELETE FROM Enrollments
WHERE enrollment_id = 5;

-- Verify changes
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Terminal Output (VS Code Execution):

```
mysql> UPDATE Students SET email = 'arjun_new@gmail.com' WHERE student_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM Enrollments WHERE enrollment_id = 5;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Students;
+------------+-------+---------------------+
| student_id | name  | email               |
+------------+-------+---------------------+
|          1 | Arjun | arjun_new@gmail.com  |
|          2 | Meera | meera@gmail.com      |
|          3 | Rahul | rahul@gmail.com      |
+------------+-------+---------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollments;
+---------------+------------+-----------+-----------------+
| enrollment_id | student_id | course_id | enrollment_date |
+---------------+------------+-----------+-----------------+
|             1 |          1 |       101 | 2026-11-01      |
|             2 |          1 |       102 | 2026-11-01      |
|             3 |          2 |       103 | 2026-11-02      |
|             4 |          3 |       101 | 2026-11-02      |
+---------------+------------+-----------+-----------------+
```

```
4 rows in set (0.00 sec)
```

Justification & Technical Explanation:
Executed standard Data Manipulation Language (DML) commands: an UPDATE query modified student 1's email with precise WHERE targeting, and a targeted DELETE statement safely pruned enrollment record 5. SELECT queries confirm that the data modifications persisted.

Task 4 – Execute Aggregate Queries

Question / Objective:
Execute Aggregate Queries to count students per course and filter courses with >1 student using GROUP BY and HAVING.

AI Prompt:
"Generate SQL aggregate queries to count the number of students enrolled in each course and display courses having more than one student enrolled using GROUP BY, COUNT(), and HAVING clauses."

Source Code:

```
-- Count students per course
SELECT c.course_name,
       COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

-- Filter courses with more than 1 student enrolled
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Terminal Output (VS Code Execution):

```
mysql> SELECT c.course_name, COUNT(e.student_id) AS student_count ...;
+------------------+---------------+
| course_name      | student_count |
+------------------+---------------+
| Database Systems |             2 |
| Data Structures  |             1 |
| Operating Systems |            1 |
+------------------+---------------+
3 rows in set (0.00 sec)

mysql> ... GROUP BY c.course_name HAVING COUNT(e.student_id) > 1;
+-----------------+
| course_name     |
+-----------------+
| Database Systems |
+-----------------+
1 row in set (0.00 sec)
```

Justification & Technical Explanation:
Aggregate function COUNT() combined with GROUP BY aggregates student enrollments per course. The HAVING

clause applies a post-aggregation filter (`COUNT(e.student_id) > 1`), accurately identifying high-enrollment courses like 'Database Systems' without violating SQL execution order rules.

Task 5 – Query Optimization

Question / Objective:
Analyze given SQL subquery and rewrite using JOINs for better performance.

AI Prompt:
"Analyze the given SQL subquery and rewrite it using JOINs for better query performance. Ensure both queries return the same results."

Source Code:

```
-- Original Subquery:
SELECT * FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 101
);

-- Optimized Query using JOIN:
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Terminal Output (VS Code Execution):

```
mysql> -- Subquery Output:
+------------+-------+---------------------+
| student_id | name  | email               |
+------------+-------+---------------------+
|          1 | Arjun | arjun_new@gmail.com |
|          3 | Rahul | rahul@gmail.com     |
+------------+-------+---------------------+
2 rows in set (0.00 sec)

mysql> -- JOIN Output:
+------------+-------+---------------------+
| student_id | name  | email               |
+------------+-------+---------------------+
|          1 | Arjun | arjun_new@gmail.com |
|          3 | Rahul | rahul@gmail.com     |
+------------+-------+---------------------+
2 rows in set (0.00 sec)
```

Justification & Technical Explanation:
The nested subquery with `IN` can force the database engine to execute correlated scans or generate temporary unindexed materializations. Rewriting it as an explicit INNER JOIN allows MySQL's query optimizer to leverage indexes on primary and foreign keys, resulting in a single execution plan with lower I/O cost while guaranteeing identical results.